The Journal of Open Source Software

pykep: A research toolbox for space flight mechanics and interplanetary trajectory design

Dario Izzo [1]

1 European Space Agency's Advanced Concepts Team, The Netherlands

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

pykep is a Python research library covering a broad range of space flight mechanics problems, with its current emphasis on the preliminary analysis of interplanetary spacecraft trajectories. It provides the mathematical building blocks for orbit propagation, orbital element conversions, Lambert arc solvers, gravity-assist flyby modelling, and both direct and indirect low-thrust transcriptions. Given a mission scenario—a spacecraft departing from one body in the solar system and arriving at another, possibly via intermediate gravity-assist flybys and with low-thrust propulsion phases—pykep allows researchers to compute transfer costs (in terms of propellant mass and time of flight), set up the corresponding optimization problems, and interface with state-of-the-art numerical optimizers. The library's C++ core can evaluate thousands of candidate trajectories per second, enabling the large-scale search over launch dates, flyby sequences, and thrust profiles that is essential during the earliest stages of mission design. pykep is not an operational flight dynamics tool; it is a research instrument designed to let scientists and engineers prototype new ideas and push the frontier of what is feasible across the full landscape of space flight mechanics problems.

Statement of Need

The preliminary design of an interplanetary mission is fundamentally an exploration problem: analysts must survey a vast combinatorial space of departure dates, planetary flyby sequences, and propulsion strategies to identify the most promising candidates before committing to costly detailed analysis. Certified tools for operational flight dynamics prioritize physical fidelity and validated models over the speed and composability needed to iterate over millions of trajectory options rapidly.

pykep was developed at the European Space Agency's Advanced Concepts Team (ACT) specifically to fill this gap. Its target audience is researchers and scientists working on astrodynamics, orbital mechanics, space flight mechanics, and related computational fields who need a flexible, Pythonic toolkit to prototype algorithms, benchmark new methods, and conduct large-scale parametric studies. While the library's current focus lies on the preliminary analysis of interplanetary trajectories, its scope is deliberately broader: the same core primitives—propagators, element conversions, and trajectory leg models—are equally applicable to Earth-orbit problems, lunar transfers, and other space flight mechanics domains. The library is explicitly *not* intended for operational mission design or for use in certified flight dynamics pipelines; it is a research tool for pushing the frontier of what is computable at the earliest stages of mission analysis.

pykep deliberately exposes its mathematical primitives—Lambert arc solvers, low-thrust trajectory legs, orbital element conversions—as first-class objects that can be composed freely and embedded in optimization loops. By integrating natively with `pagmo` (Biscani & Izzo, 2020) (a parallel global optimization library) and `Heyoka` (Biscani & Izzo, 2021) (a Taylor

The Journal of Open Source Software

integration suite), pykep provides a full stack from ephemeris evaluation to global trajectory search within a single Python environment—a coherent collection of well-defined, composable primitives ready to be assembled into novel research workflows. The many algorithms available in the library are largely the result of original research carried out at the ACT over more than a decade, complemented by implementations of best-in-class methods from the broader astrodynamics literature.

State of the Field

Several open-source tools address related aspects of spacecraft trajectory design, and it is important to articulate how pykep relates to and differs from each.

Orekit (Maisonobe et al., 2010) is a Java-based orbital mechanics library (with a Python facade) targeted at operational applications. It provides high-fidelity force models and validated propagators, making it the tool of choice when a precise, certified answer to a specific trajectory question is required. Its design is not optimized for the rapid, repeated evaluations over large search spaces that characterize preliminary mission design.

GMAT (Hughes et al., 2014) is NASA's General Mission Analysis Tool, a comprehensive high-fidelity simulator for a wide range of mission types. It is primarily a GUI- and script-driven application oriented toward operational analysis rather than algorithmic exploration and programmatic access.

GPOPS-II (Patterson & Rao, 2014) is a MATLAB-based optimal control software using hp-adaptive Gaussian quadrature collocation. It excels at locally converging a single, complex continuous trajectory but is proprietary and not designed for global search over mission scenarios or large parameter sweeps.

poliastro (Cano Rodríguez et al., 2022) is a Python library for general orbital mechanics. It shares some features with pykep (orbit propagation, Lambert solvers) but is oriented toward education and general orbital analysis rather than the specialized requirements of preliminary interplanetary mission design, specifically multi-gravity-assist sequences, low-thrust transcriptions, and tight integration with global optimization algorithms.

pykep occupies a distinct niche by combining, in a single research-oriented Python package: fast multi-revolution Lambert solvers (Izzo, 2015); both direct (Sims-Flanagan (Sims & Flanagan, 1997), zero-order-hold (Izzo et al., 2026)) and indirect (Pontryagin-based) low-thrust transcriptions; native coupling with `pagmo` for global and multi-objective optimization; Taylor-based high-accuracy propagation via `Heyoka` (Biscani & Izzo, 2021, 2022); and a growing benchmark suite (`gym`) enabling reproducible comparisons of trajectory optimization algorithms. No existing open-source package provides this combination in a research-grade Python library explicitly designed to support the development of novel algorithms.

Software Design

pykep 3 represents a complete redesign of the library from its earlier Python 2 origins. The computational core is implemented in modern C++23 and exposed to Python via `pybind11`. This architecture ensures that the innermost evaluation loops—Lambert arcs, Sims-Flanagan mismatch constraints (Sims & Flanagan, 1997), zero-order-hold propagation (Izzo et al., 2026), orbital element conversions (Walker et al., 1985)—run at native speed while remaining fully accessible from Python.

A central abstraction is the *user-defined planet-like object* (`udpla`): any object that provides position and velocity as a function of epoch can serve as a trajectory endpoint or flyby body, regardless of whether it is backed by SPICE kernels, the SGP4 model, an analytic Keplerian approximation, or a user-supplied function. This makes all higher-level trajectory solvers

ephemeris-agnostic and straightforward to test with analytical models before switching to full SPICE data.

A deliberate architectural choice that runs throughout the library is the complete avoidance of inheritance. Rather than requiring users to subclass abstract base classes—a pattern that introduces coupling, fragile hierarchies, and verbose boilerplate, pykep relies on *type erasure*: any object that satisfies a documented concept (i.e., exposes the right methods with the right signatures) is accepted by the corresponding algorithm or container, regardless of its type. This gives rise to the pervasive *user-defined* vocabulary in the API: `udpla` (user-defined planet-like object), `udp` (user-defined problem), `uda` (user-defined algorithm), and so on. Each prefix signals that the corresponding slot in the system accepts any conforming type, not a prescribed class hierarchy. The result is an interface that is simultaneously extensible, a researcher can plug in a custom ephemeris, propagator, or objective function with a handful of Python lines—and free from the cognitive overhead of classical object-oriented inheritance patterns.

Trajectory optimization problems are exposed as *user-defined problem* (`udp`) objects compatible with `pagmo` (Biscani & Izzo, 2020), so that any optimizer in the `pagmo` ecosystem (e.g. differential evolution, particle swarm, self-adaptive evolution strategies, and branch-and-bound) can be applied without modification. The design separates the mathematical primitives (C++) from the optimization problem definitions (Python/C++) and auxiliary tools such as visualization and low-thrust approximations (Hennes et al., 2016) (pure Python). This layering keeps the library maintainable and makes it straightforward for researchers to contribute new trajectory models or problem formulations. A set of benchmark problems (`gym`), modeled on challenges from the Global Trajectory Optimization Competitions (Izzo et al., 2016), is included to support reproducible algorithm development and direct comparison of competing approaches.

Research Impact Statement

pykep and its predecessors have been the primary computational tool of the ACT for over a decade of competition-level and research-grade interplanetary trajectory work. The library was instrumental in the ACT's participation in multiple editions of the Global Trajectory Optimization Competition (GTOC) (Izzo et al., 2013, 2016, 2025)—a series of open benchmarks widely regarded as among the most demanding trajectory optimization problems in the field, and a venue where the library has consistently enabled top-tier results against leading astrodynamics teams worldwide.

Beyond competition, pykep has underpinned preliminary mission analysis for several ESA concepts, including the Hera asteroid deflection mission, the M-ARGO interplanetary CubeSat, and early feasibility studies for the Titan and Enceladus Mission (TandEM) and the Laplace mission to the outer solar system. It has supported peer-reviewed research on novel indirect methods via surrogate primer vectors (Beauregard et al., 2024), comparative studies of machine-learning and astrodynamical approaches to low-thrust transfers (Acciarini et al., 2024), approximation methods for asteroid-belt transfer cost estimation (Hennes et al., 2016), and high-order Taylor methods for astrodynamics (Biscani & Izzo, 2021, 2022).

The library is distributed under an open-source license, available on PyPI and conda-forge, and is actively maintained. Its documentation includes tutorials covering basic propagation, Lambert problems, multi-gravity-assist trajectory optimization, and low-thrust mission design, lowering the barrier for adoption by the broader research community.

AI Usage Disclosure

AI-assisted writing tools (GitHub Copilot) were used in an assistive capacity during the drafting of this paper. All technical content, bibliographic references, and scientific claims were reviewed

and verified by the authors.

Acknowledgements

The authors thank the many ACT researchers and interns who have used and improved pykep over the years. Special thanks are due to Francesco Biscani in particular for the co-development of `pagmo` and `Heyoka`, his early interest in pykep and to the open-source communities behind `pybind11` and `spiceypy`. Development has been supported by the European Space Agency through the Advanced Concepts Team's internal research programme.

References

Acciarini, G., Beauregard, L., & Izzo, D. (2024). Computing low-thrust transfers in the asteroid belt, a comparison between astrodynamical manipulations and a machine learning approach. *arXiv Preprint arXiv:2405.18918*.

Beauregard, L., Izzo, D., & Acciarini, G. (2024). Breaking traditions: introducing a surrogate Primer Vector in non Keplerian dynamics. *arXiv Preprint arXiv:2405.18903*.

Biscani, F., & Izzo, D. (2020). A parallel global multiobjective framework for optimization: pagmo. *Journal of Open Source Software*, *5*(53), 2338.

Biscani, F., & Izzo, D. (2021). Revisiting high-order taylor methods for astrodynamics and celestial mechanics. *Monthly Notices of the Royal Astronomical Society*, *504*(2), 2614–2628.

Biscani, F., & Izzo, D. (2022). Reliable event detection for taylor methods in astrodynamics. *Monthly Notices of the Royal Astronomical Society*, *513*(4), 4833–4844.

Cano Rodríguez, J. L., Dorado, J. M., Bello Marín, D., & others. (2022). Poliastro: An astrodynamics library written in python. *Journal of Open Source Software*, *7*(78), 4742. https://doi.org/10.21105/joss.04742

Hennes, D., Izzo, D., & Landau, D. (2016). Fast approximators for optimal low-thrust hops between main belt asteroids. *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1–7.

Hughes, S. P., Qureshi, R. H., Cooley, D. S., & Parker, J. S. (2014). Verification and validation of the General Mission Analysis Tool (GMAT). *AIAA/AAS Astrodynamics Specialist Conference*, 4151.

Izzo, D. (2015). Revisiting lambert's problem. *Celestial Mechanics and Dynamical Astronomy*, *121*, 1–15.

Izzo, D., Hennes, D., Simões, L. F., & Märtens, M. (2016). Designing complex interplanetary trajectories for the global trajectory optimization competitions. *Space Engineering: Modeling and Optimization with Case Studies*, 151–176.

Izzo, D., Holt, H., Acciarini, G., Beauregard, L., & Shimane, Y. (2026). A practical guide to implementing generic zero-hold interplanetary trajectory legs. *Proceedings of the 30th International Symposium on Space Flight Dynamics (ISSFD)*.

Izzo, D., Märtens, M., Beauregard, L., Bannach, M., Acciarini, G., Blazquez, E., Hadjiivanov, A., Grover, J., Heißel, G., Shimane, Y., & others. (2025). Asteroid mining: ACT&Friends' results for the GTOC12 problem. *Astrodynamics*, *9*(1), 19–40.

Izzo, D., Simoes, L. F., Martens, M., Croon, G. C. de, Heritier, A., & Yam, C. H. (2013). Search for a grand tour of the jupiter galilean moons. *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation*, 1301–1308.

Maisonobe, L., Parraud, P., & Vehil, R. (2010). Orekit: An open source library for

operational flight dynamics applications. *Proceedings of the 4th International Conference on Astrodynamics Tools and Techniques (ICATT)*.

Patterson, M. A., & Rao, A. V. (2014). GPOPS-II: A MATLAB software for solving multiple-phase optimal control problems using hp-adaptive Gaussian quadrature collocation methods and sparse nonlinear programming. *ACM Transactions on Mathematical Software*, *41*(1), 1–37. https://doi.org/10.1145/2558904

Sims, J. A., & Flanagan, S. N. (1997). Preliminary design of low-thrust interplanetary missions. *Paper AAS 99-338, AAS/AIAA Astrodynamics Specialist Conference, Girdwood, Alaska, August 16-18, 1999*.

Walker, M. J., Ireland, B., & Owens, J. (1985). A set modified equinoctial orbit elements. *Celestial Mechanics*, *36*(4), 409–419.